SESION

MAY 2026 (WEEKEND)

BIT1034

Advance Programming.

Assignment

Lecturer:

DR. FIONA

NO	NAME	PROGRAM	STUDENT ID
1	MOHD SAHIFFUL AZHAR BIN ABD SUKOR	BIT	B24120060

SMART GAMING TOURNAMENT ANALYTICS SYSTEM

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABSTRACT

This project focuses on developing a Smart Gaming Tournament Analytics System using Python and Streamlit. The main purpose of the system is to help tournament organizers manage player data, match results, rankings, and analytics in a more efficient way. Many small gaming tournaments still use manual methods such as spreadsheets and paper records. This can cause data errors, slow ranking updates, and difficulty in analyzing player performance.

The proposed system uses Streamlit to build a simple web dashboard, Python to process data, Pandas to manage CSV datasets, SQLite to store data, and Matplotlib or Plotly to display charts. The system provides functions such as player registration, CSV upload, automatic ranking generation, and analytics visualization. Agile methodology is used because it allows the project to be developed step by step and improved based on feedback.

The repaired report also includes detailed implementation, testing cases, interface screenshots or mockups, GitHub submission checklist, and project appendices. Overall, this project improves tournament management, reduces manual work, and provides a user-friendly analytics dashboard for gaming tournament organizers.

Keywords: Python, Streamlit, SQLite, Gaming Tournament, Analytics Dashboard, CSV Dataset, Ranking System

1. INTRODUCTION

Technology has changed the way gaming tournaments are organized. Today, many gaming competitions are managed online because players can join from different locations. Popular games such as Mobile Legends, PUBG, Valorant, FIFA, and Call of Duty often use online tournament systems to manage players, match results, and rankings.

However, many small tournament organizers still use manual methods such as spreadsheets, paper forms, or handwritten records. This method is not efficient when the number of players becomes large. It also increases the chance of human error, especially when updating scores and calculating rankings.

This project proposes a Smart Gaming Tournament Analytics System using Python and Streamlit. The system is designed as a simple web-based dashboard for managing player information, importing CSV tournament data, storing data in SQLite, generating rankings automatically, and viewing analytics charts.

The project is suitable for the BIT1034 Advance Programming course because it applies programming concepts such as database connection, CSV processing, data visualization, web application development, and basic software testing (Python Software Foundation, 2024; Streamlit Inc., 2024). The system is also designed using simple modules so it is easier to maintain and improve in the future.

2. PROBLEM STATEMENT

- Manual data management using spreadsheets or paper records is slow and can cause mistakes.
- Ranking updates are difficult because points and results must be calculated manually after every match.
- Tournament organizers cannot easily analyze player performance such as wins, losses, win rate, and total points.

- Data searching becomes slower when there are many players and match records.
- Some existing tournament systems are expensive or too complicated for small organizers.
- There is no simple dashboard that combines CSV upload, database storage, automatic ranking, and analytics charts.

Because of these problems, a low-cost and user-friendly analytics system is needed to improve tournament management and support better decision making.

3. OBJECTIVES

3.1 General Objective

To develop a Smart Gaming Tournament Analytics System using Python and Streamlit for efficient gaming tournament management and player analytics.

3.2 Specific Objectives

No.	Objective	Related Output
1	To develop a web-based tournament management dashboard using Streamlit.	Dashboard
2	To integrate SQLite database for storing player, tournament, match, and ranking data.	Database tables
3	To import and process tournament data from CSV datasets.	CSV upload feature
4	To generate automatic player rankings based on points, wins, and match performance.	Ranking table
5	To display tournament analytics using charts and dashboard components.	Charts and graphs
6	To test the system functions and confirm that the project requirements are achieved.	Testing table
7	To prepare a GitHub repository and submission checklist for project delivery.	Repository link and checklist

4. LITERATURE REVIEW / BACKGROUND

4.1 Online Gaming Tournament Systems

Online gaming tournament systems help organizers manage registration, match scheduling, scoring, and rankings. These systems are important because tournament data changes quickly during competitions. A proper system helps reduce manual work and improves result accuracy (Sommerville, 2020).

4.2 Python Programming Language

Python is suitable for this project because the syntax is easy to understand and it supports many libraries for data processing, database connection, and visualization (Python Software Foundation, 2024; McKinney, 2022). Python is beginner-friendly for students but powerful enough for real software projects.

4.3 Streamlit Framework

Streamlit is a Python framework used to build web applications quickly (Streamlit Inc., 2024). It is suitable for data dashboard projects because it can display tables, charts, upload widgets, and simple navigation with less frontend coding.

4.4 SQLite Database

SQLite is a lightweight database that can store structured data without requiring a separate database server (SQLite Consortium, 2024). It is suitable for small and medium projects because it is simple to configure and works well with Python.

4.5 CSV Dataset Processing

CSV files are easy to create, edit, and share. In this project, CSV upload allows tournament organizers to import player statistics and match results quickly without typing all data manually.

4.6 Data Visualization

Data visualization helps users understand tournament performance more clearly. Charts can show player rankings, win and loss comparison, points distribution, and testing results (Pandas Development Team, 2024; Hunter, 2023; Plotly Technologies Inc., 2024). This makes the system more useful than a normal data entry application.

5. SYSTEM DESIGN AND FLOWCHART

System design explains how the application works and how users interact with the system. The system uses a modular design because each module has a clear function. This makes the project easier to develop, test, and maintain.

5.1 Main System Modules

Module	Main Function
Login Module	Controls access to the system using username and password.
Tournament Management Module	Allows admin to create and update tournament details.
Player Management Module	Stores and updates player information.
CSV Upload Module	Imports player and match data from CSV files.
Ranking Module	Calculates ranking based on wins, scores, and points.
Analytics Dashboard Module	Displays charts, tables, and tournament summaries.
Report Export Module	Allows the organizer to export or view tournament reports.

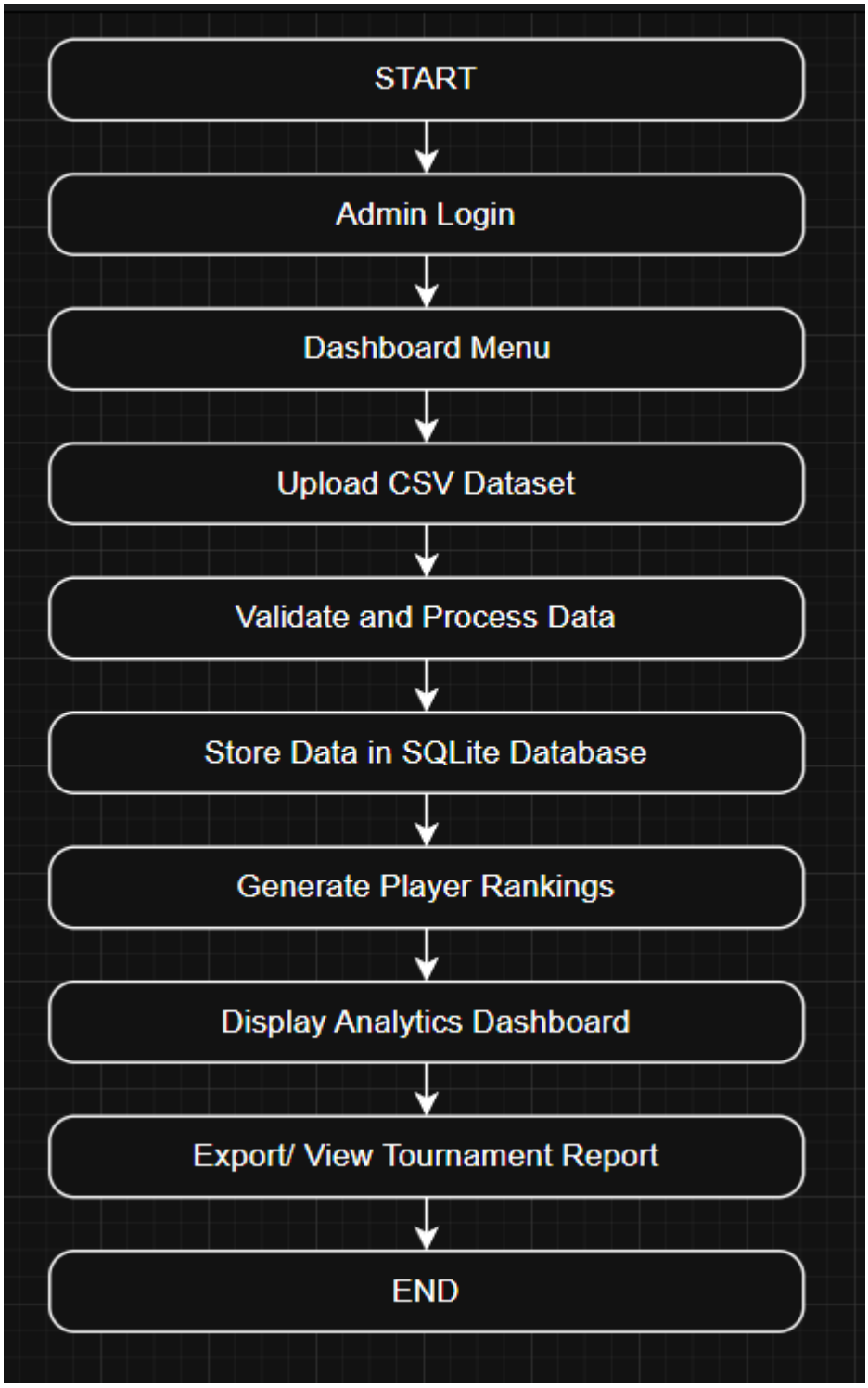


Figure 5.1: System Flowchart

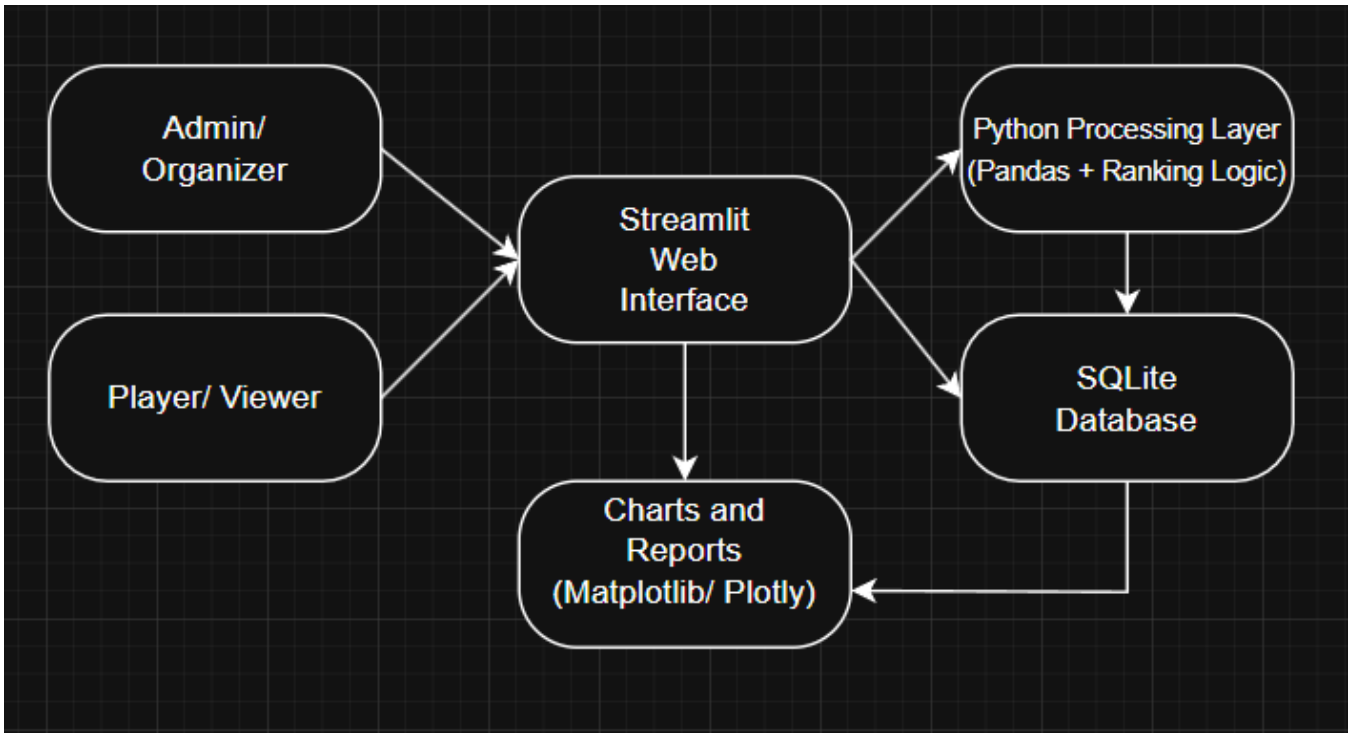


Figure 5.2: Proposed System Architecture

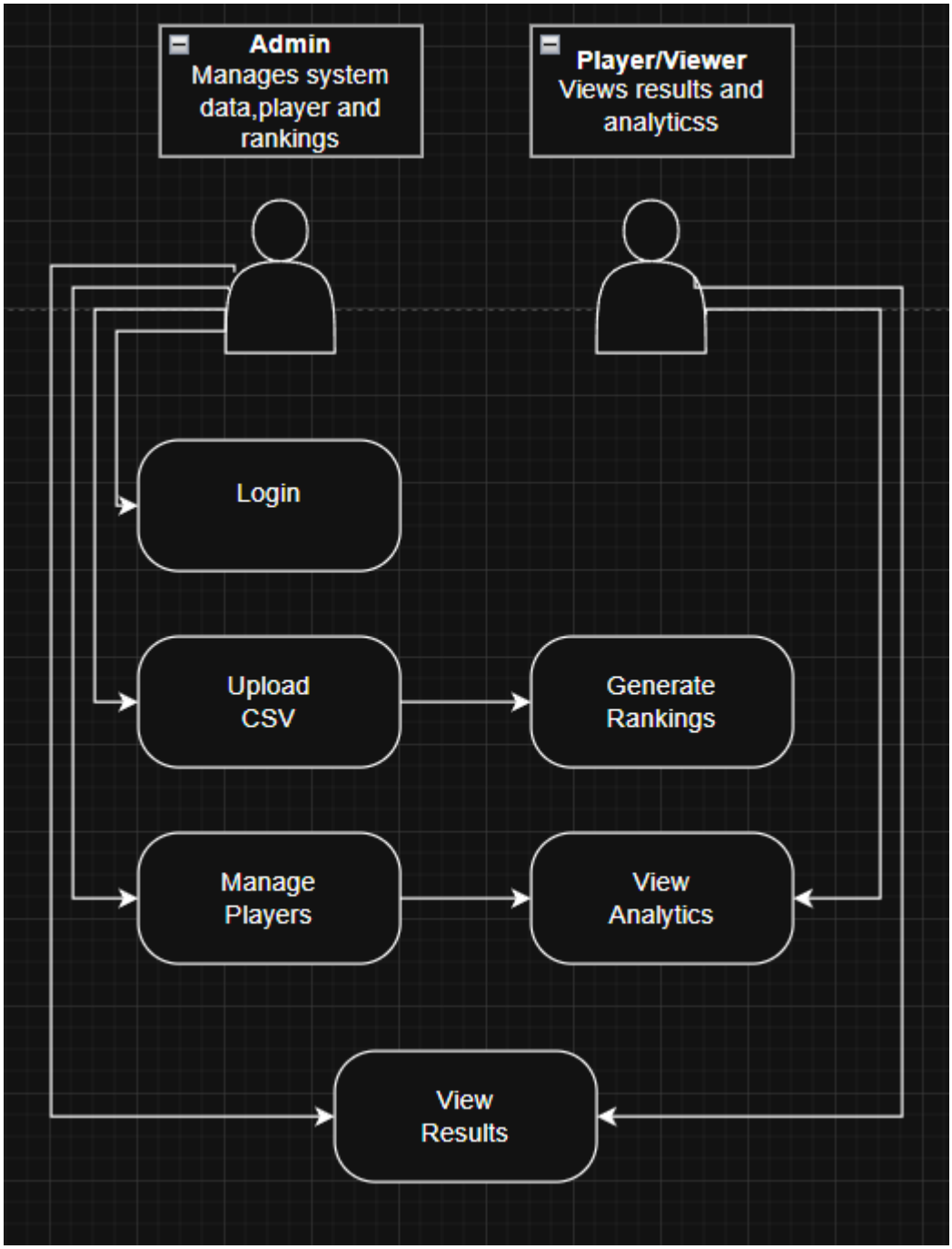


Figure 5.3: Use Case Diagram

5.2 Use Case Explanation

The admin can login, upload CSV files, manage players, manage tournaments, generate rankings, view analytics, and export reports. Normal users can view rankings, search player information, and view tournament results. The use case diagram separates admin functions and normal user functions clearly.

6. DATABASE DESIGN

The system uses SQLite database to store tournament data in organized tables. The database contains Players, Tournaments, Matches, and Rankings tables. This structure reduces duplicated data and makes searching easier.

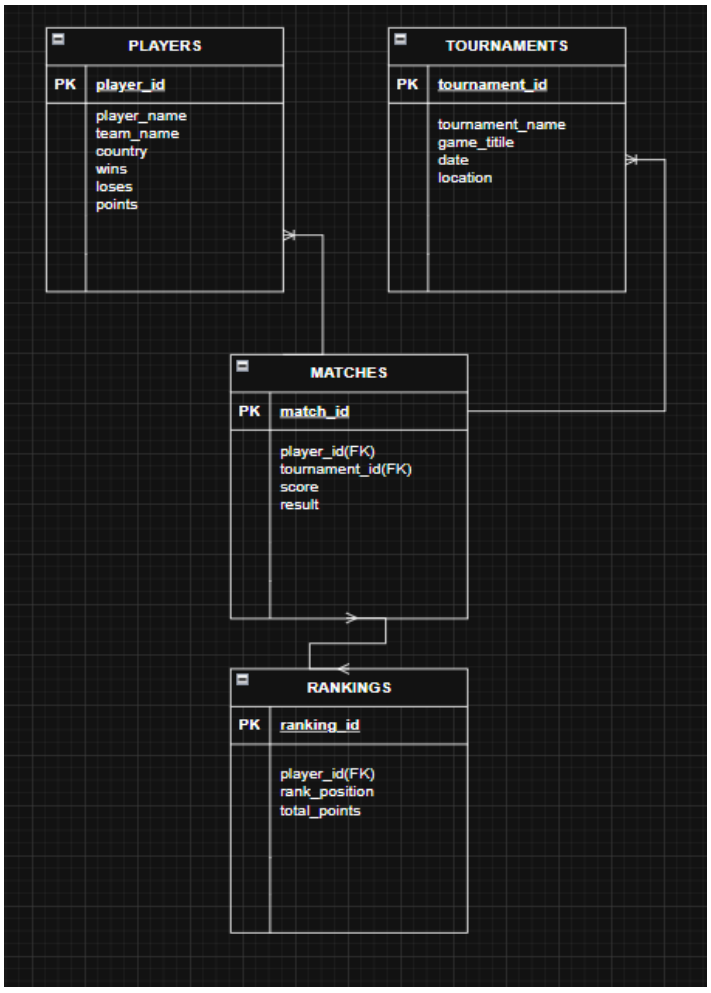


Figure 6.1: Entity Relationship Diagram

Field Name	Data Type	Description
player_id	INTEGER	Primary key
player_name	TEXT	Player name
team_name	TEXT	Team name
country	TEXT	Player country
wins	INTEGER	Total wins
losses	INTEGER	Total losses
points	INTEGER	Tournament points

Table 6.1: Players Table Structure

Field Name	Data Type	Description
tournament_id	INTEGER	Primary key
tournament_name	TEXT	Tournament name
game_title	TEXT	Game title
date	TEXT	Tournament date
location	TEXT	Tournament location

Table 6.2: Tournaments Table Structure

Field Name	Data Type	Description
match_id	INTEGER	Primary key
player_id	INTEGER	Foreign key
tournament_id	INTEGER	Foreign key
score	INTEGER	Match score
result	TEXT	Win or loss result

Table 6.3: Matches Table Structure

Field Name	Data Type	Description
ranking_id	INTEGER	Primary key
player_id	INTEGER	Foreign key
rank_position	INTEGER	Player rank
total_points	INTEGER	Total points

Table 6.4: Rankings Table Structure

6.5 Database Relationship Explanation

- One player can join many matches.
- One tournament can contain many matches.
- The ranking table uses player performance data to generate ranking positions.
- The database helps reduce duplicate data and improves data searching.

7. METHODOLOGY

This project uses Agile methodology because it supports flexible and continuous development. Agile methodology supports flexible and continuous development because it allows the system to be improved step by step based on testing and feedback (Atlassian, 2024; Sommerville, 2020) Agile is suitable for this system because the interface, ranking formula, and dashboard design can be improved step by step after testing.

Agile Phase	Activity in This Project	Output

Requirement Analysis	Identify needed functions such as CSV upload, database, ranking, and dashboard.	Requirement list
System Design	Prepare flowchart, database tables, and system modules.	Design diagrams
Development	Code the system using Python, Streamlit, SQLite, Pandas, and visualization libraries.	Working prototype
Testing	Test login, CSV upload, database storage, ranking generation, and dashboard display.	Testing results
Deployment	Run the Streamlit application locally or on a hosting platform.	Usable system
Maintenance	Fix bugs and improve features based on feedback.	Improved system version

7.1 Development Environment

Component	Description
Programming Language	Python 3.x
Framework	Streamlit
Database	SQLite
Data Processing Library	Pandas
Visualization Libraries	Matplotlib and Plotly
Version Control	Git and GitHub
Code Editor	Visual Studio Code or similar IDE

7.2 Kanban Board for Task Management

Besides Agile methodology, this project also uses a simple Kanban approach to manage development tasks. Kanban helps organize work into three stages: To Do, In Progress, and Done. This makes it easier to monitor project progress and ensure that all required functions are completed.

For this project, tasks such as database design, CSV upload, Streamlit dashboard, ranking generation, testing, screenshots, GitHub upload, and report writing were managed using a Kanban board. This approach supports Agile development because the system can be improved step by step based on task priority and feedback.

To Do	In Progress	Done
Design database	Develop CSV upload	Dashboard completed
Prepare flowchart	Create ranking logic	Testing completed
Write report	Add charts	GitHub uploaded

Table 7.2: Kanban Task Management

To Do:

- Design database structure

- Prepare CSV dataset

- Create system flowchart

- Write report content

In Progress:

- Develop Streamlit dashboard

- Create CSV upload function

- Build SQLite database connection

- Generate ranking logic

Done:

- Dashboard completed

- CSV upload completed

- Ranking page completed

- Testing completed

- Screenshots captured

- GitHub repository uploaded

7.3 DevOps Practice

This project also applies basic DevOps practice to support project delivery and maintenance. DevOps is not used as the main development methodology, but it helps in managing source code, preparing deployment files, and improving project workflow.

In this project, GitHub is used as the version control platform to store the source code and project files. The Streamlit application files such as app.py, players.csv, requirements.txt, and README.md are uploaded to GitHub so the project can be accessed, reviewed, and updated easily.

The requirements.txt file supports deployment because it lists all Python libraries needed to run the system. Testing is also done before final submission to make sure the CSV upload, database storage, ranking generation, and dashboard display work correctly.

By applying simple DevOps practice, the project becomes more organized, easier to maintain, and ready for future deployment using platforms such as Streamlit Community Cloud.

DevOps Practice	How It Applies to Your Project
Version control	Source code uploaded to GitHub
Collaboration	Group members can access same code
Deployment preparation	Streamlit files and requirements.txt prepared
Testing before release	System tested before submission
Maintenance	Future updates can be pushed to GitHub

Table 7.3: DevOps Practice

8. CODING IMPLEMENTATION

Coding implementation is the process of converting the system design into a working program. Python is used as the main programming language because it is simple, readable, and suitable for data processing and dashboard development. Python libraries such as Pandas, Matplotlib, and Plotly are useful for CSV processing, data analysis, and data visualization in dashboard systems (Pandas Development Team, 2024; Hunter, 2023; Plotly Technologies Inc., 2024).

Library	Purpose
Streamlit	Build the web-based dashboard
Pandas	Read and process CSV datasets
SQLite3	Connect and manage SQLite database
Matplotlib	Create basic charts
Plotly	Create interactive charts

8.2 CSV Upload Code Example

```
import streamlit as st
import pandas as pd

uploaded_file = st.file_uploader('Upload CSV file', type=['csv'])

if uploaded_file is not None:
    data = pd.read_csv(uploaded_file)
    st.success('CSV file uploaded successfully')
    st.dataframe(data.head())
```

8.3 SQLite Database Connection Code Example

```
import sqlite3

conn = sqlite3.connect('tournament.db')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE IF NOT EXISTS players (
    player_id INTEGER PRIMARY KEY AUTOINCREMENT,
    player_name TEXT,
```

```
    team_name TEXT,
    country TEXT,
    wins INTEGER,
    losses INTEGER,
    points INTEGER
)
''')
conn.commit()
```

8.4 Save CSV Data into Database

```
for index, row in data.iterrows():
    cursor.execute('''
        INSERT INTO players (player_name, team_name, country, wins, losses, points)
        VALUES (?, ?, ?, ?, ?, ?)
    ''', (row['Player Name'], row['Team'], row['Country'],
          row['Wins'], row['Losses'], row['Points']))

conn.commit()
st.success('Player data saved into SQLite database')
```

8.5 Ranking Generation Logic

```
players = pd.read_sql_query('SELECT * FROM players', conn)
players = players.sort_values(by=['points', 'wins'], ascending=False)
players['rank_position'] = range(1, len(players) + 1)

st.subheader('Player Rankings')
st.dataframe(players[['rank_position', 'player_name', 'team_name', 'points']])
```

8.6 Chart Generation Code Example

```
import plotly.express as px

fig = px.bar(players, x='player_name', y='points',
             title='Player Ranking by Total Points')
st.plotly_chart(fig, use_container_width=True)
```

8.7 Simple Login / Session Logic

```
if 'logged_in' not in st.session_state:
    st.session_state.logged_in = False

username = st.text_input('Username')
password = st.text_input('Password', type='password')

if st.button('Login'):
    if username == 'admin' and password == 'admin123':
        st.session_state.logged_in = True
        st.success('Login successful')
    else:
        st.error('Invalid username or password')
```

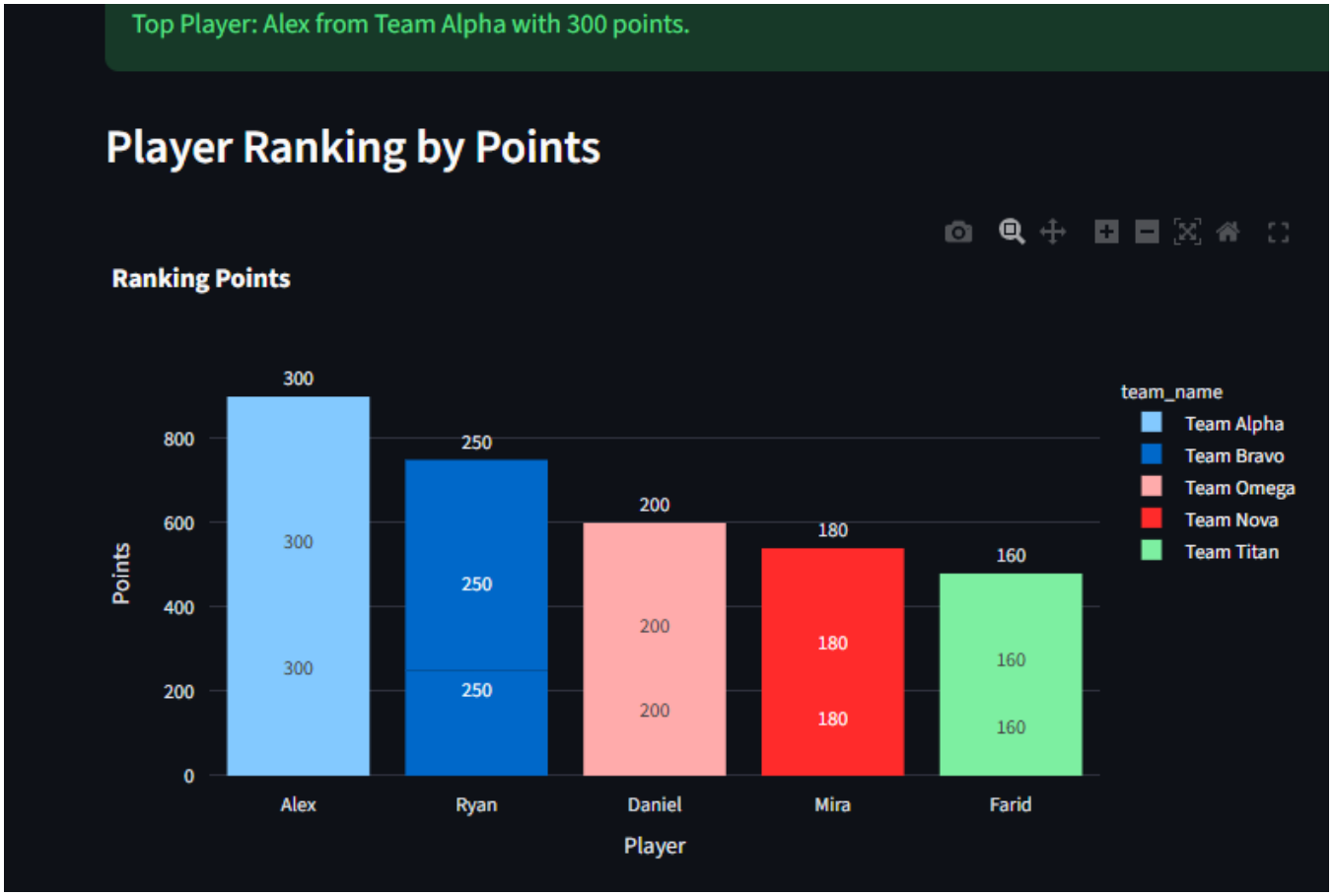


Figure 8.1: Player Ranking Chart

The ranking chart displays total points for each player. This helps organizers quickly identify the top players in the tournament.

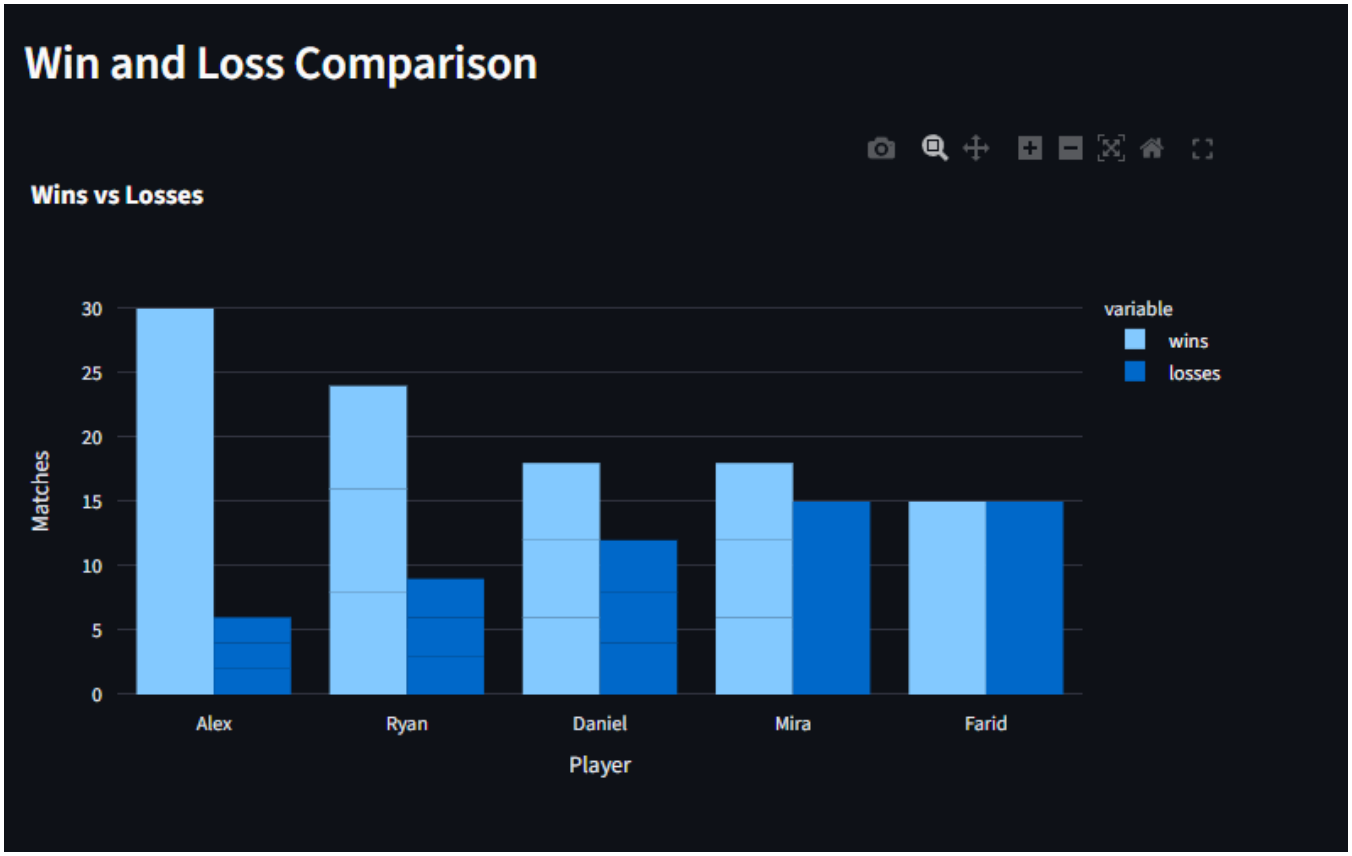
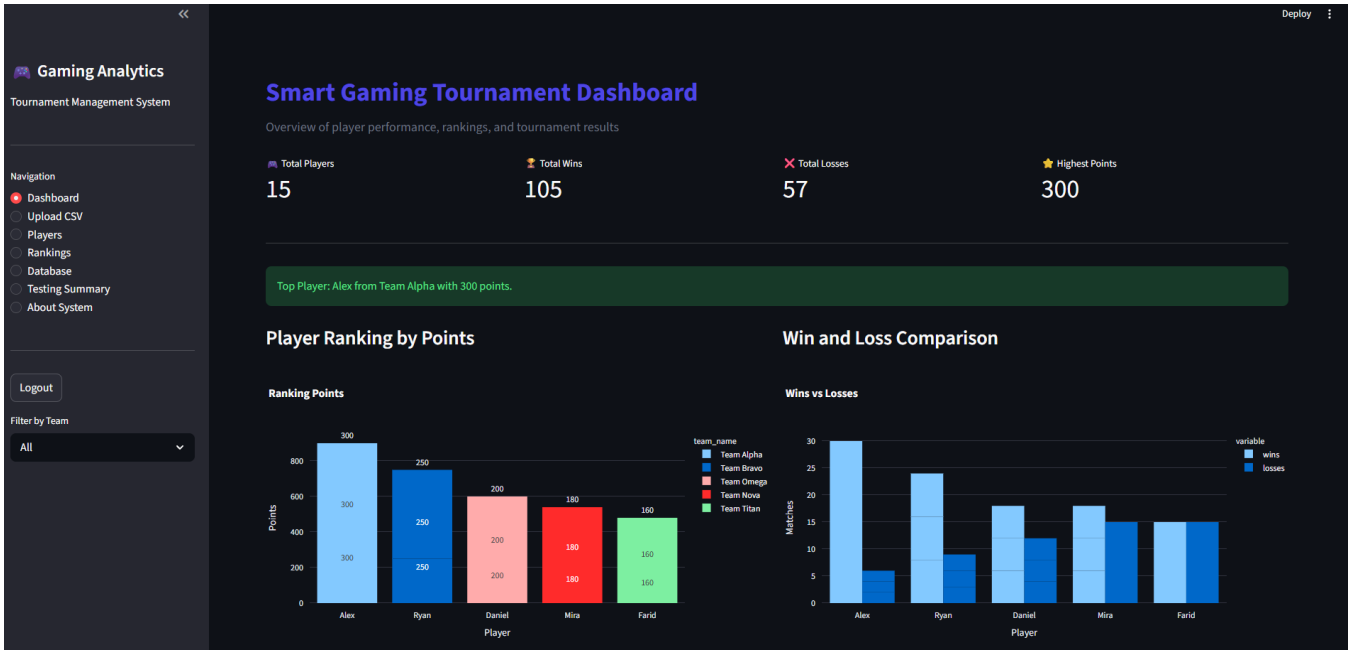


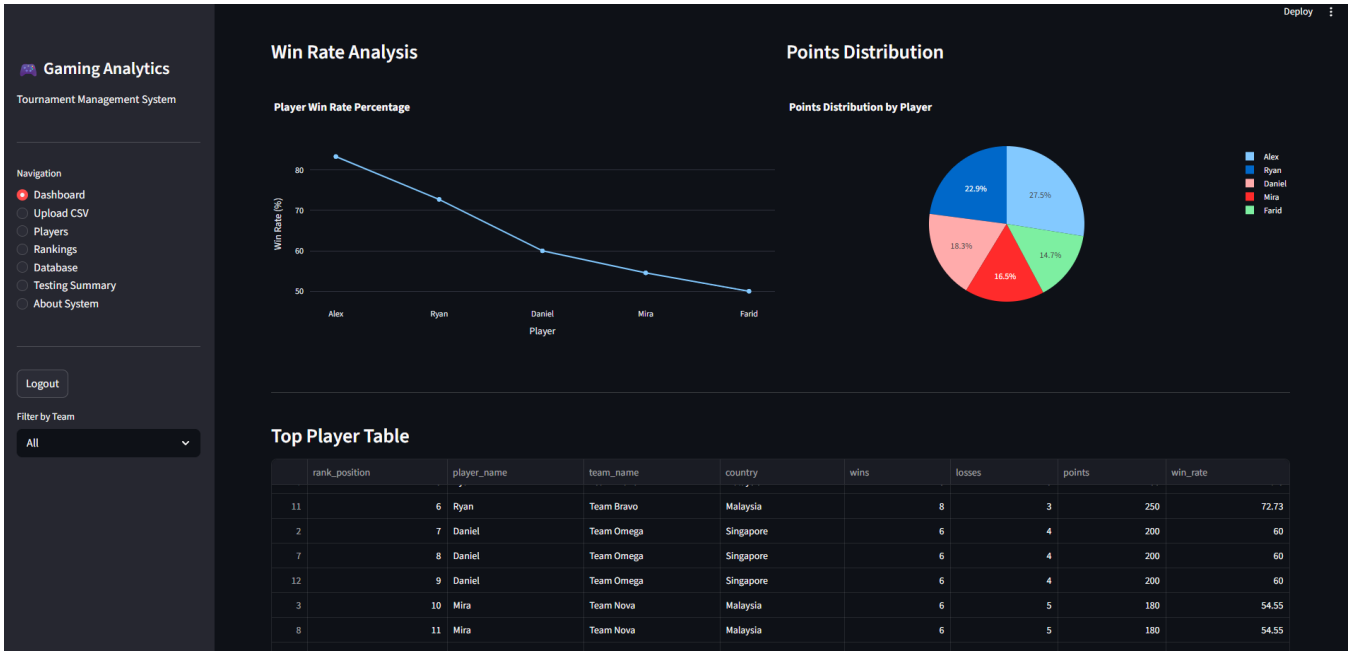
Figure 8.2: Sample Win and Loss Analysis

The win and loss analysis helps compare player performance. A player with many wins and fewer losses will normally have a stronger tournament result.

9. SYSTEM INTERFACE AND SCREENSHOTS

This section shows the actual screenshots captured from the running Streamlit application using the command streamlit run app.py. The purpose of these screenshots is to show the expected interface layout and system functions.





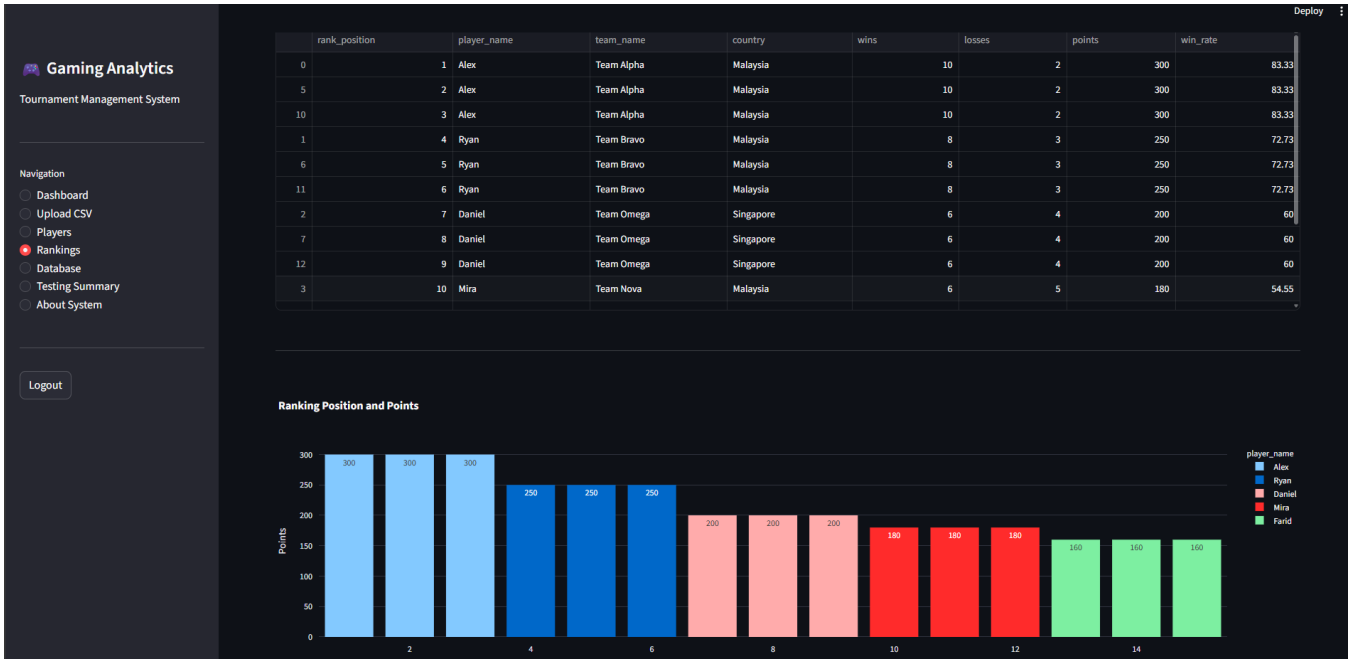


Figure 9.1: Dashboard Page

The dashboard page shows summary cards, ranking chart, and top player table. This page helps users understand tournament status quickly.

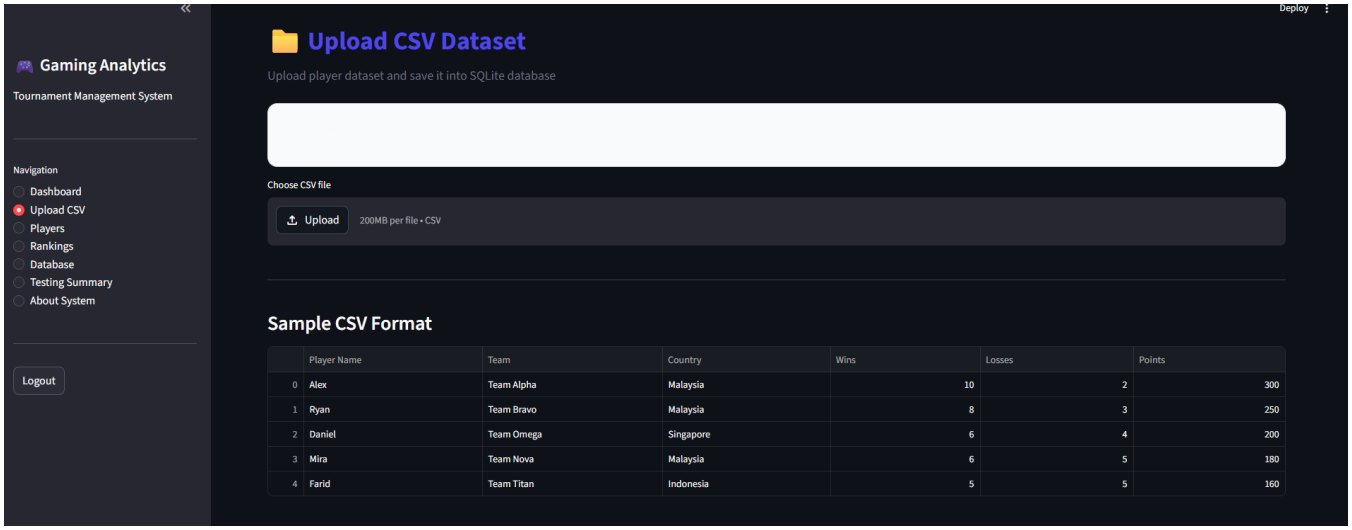


Figure 9.2: Upload CSV Page

The CSV upload page allows the admin to upload player or match datasets. The system validates data and stores it into SQLite.

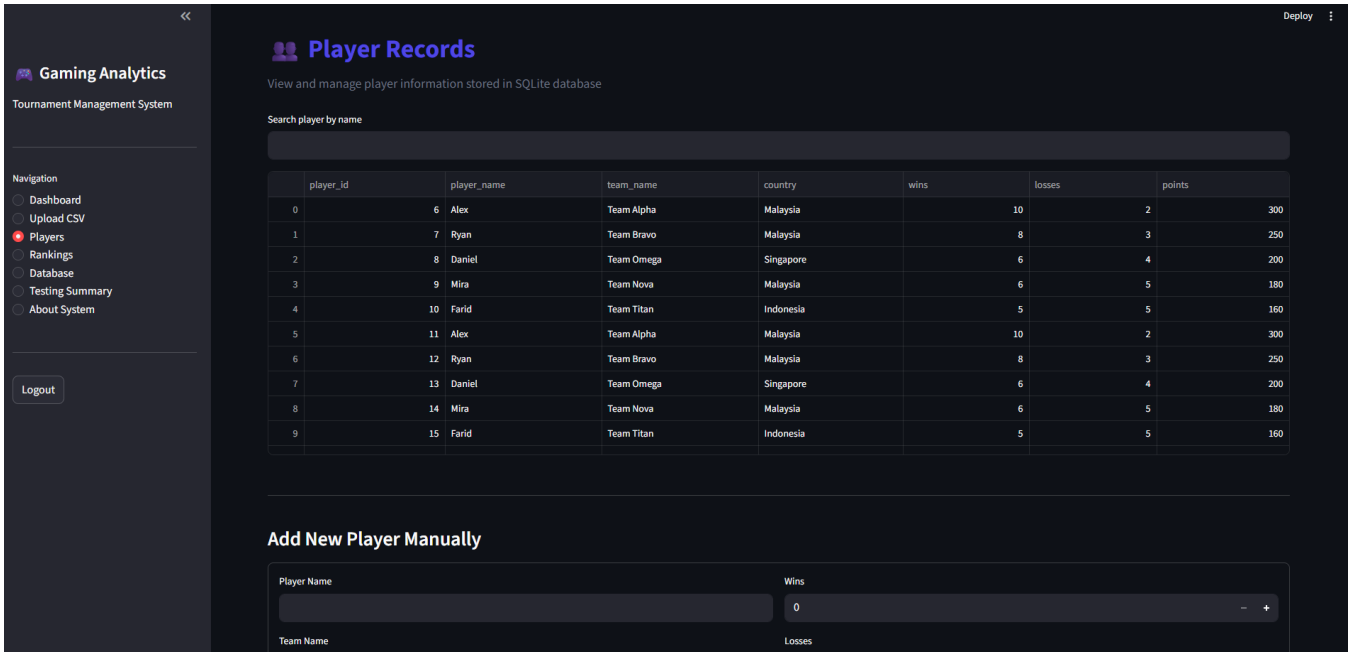


Figure 9.3: Players Page

The player management page displays player records and supports adding, updating, and searching player information.

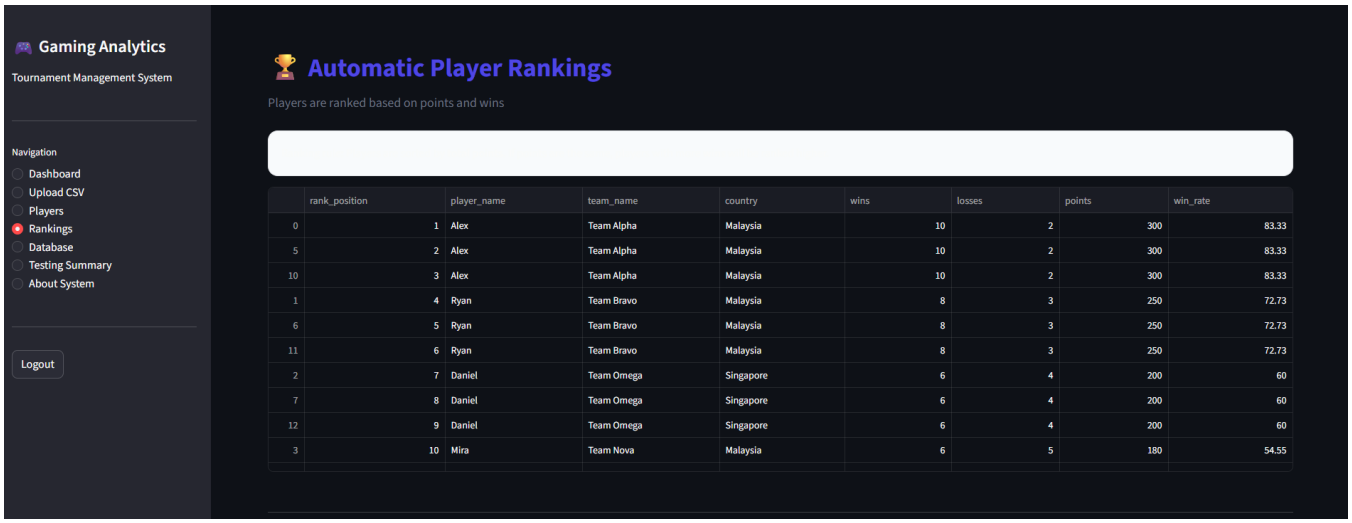


Figure 9.4: Rankings Page

The ranking page shows automatic ranking based on points, wins, and match score. This reduces manual calculation errors.

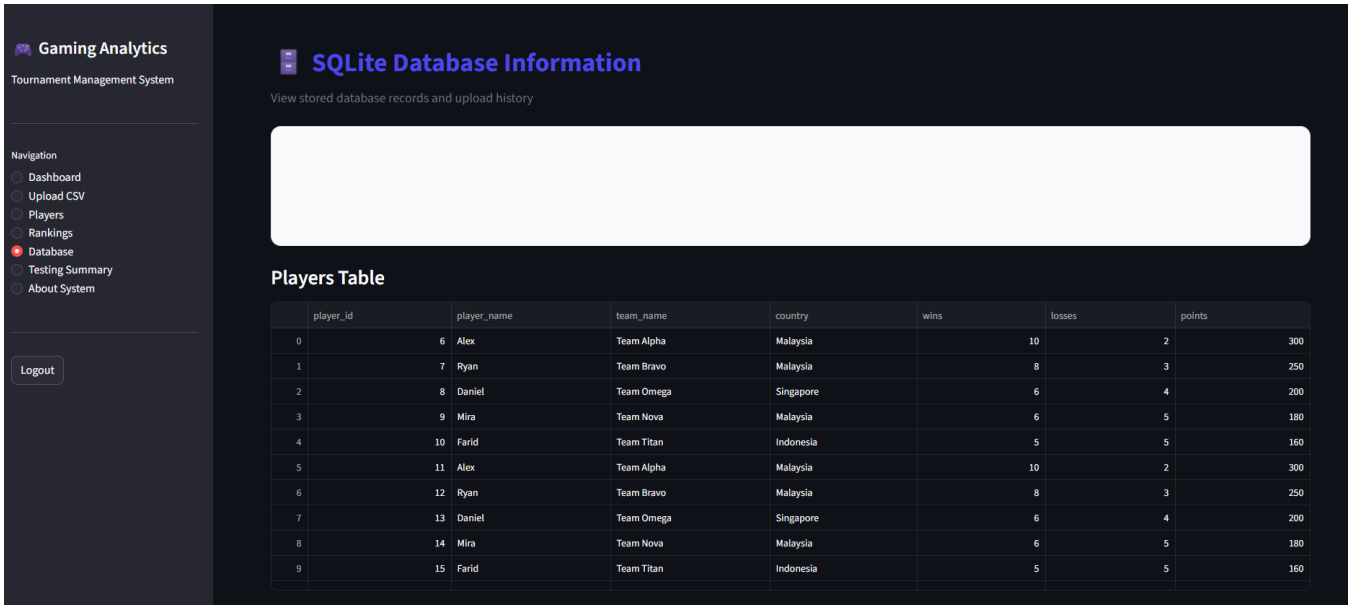


Figure 9.5: Reports Page

The reports page displays analytics output that can support tournament review and decision making.

10. TESTING AND RESULTS

Testing is important to make sure the system works correctly and meets the project objectives. The system is tested using functional testing, database testing, interface testing, invalid input testing, and simple performance testing.

Test Case	**Expected Result**	**Actual Result**	**Status**
User Login	User can login successfully	Successful	Pass
CSV Upload	CSV file imported correctly	Successful	Pass
Database Storage	Data stored in SQLite database	Successful	Pass
Ranking Generation	Ranking displayed according to points	Successful	Pass
Dashboard Display	Charts and tables displayed correctly	Successful	Pass
Report Export	Tournament report can be viewed or exported	Successful	Pass

Figure 10.1: Functional Testing Summary

10.2 Additional Testing Results

Testing Type	Scenario	Expected Output	Result
Database Testing	Insert new player record	Record saved without error	Pass
CSV Invalid File Testing	Upload file with missing Points column	System shows validation warning	Pass
Ranking Accuracy Testing	Two players have different points	Higher points appear first	Pass
Tie-breaker Testing	Two players have same points	Player with more wins appears higher	Pass
Interface Testing	Open dashboard menu	Menu and chart display correctly	Pass
Performance Testing	Process 1,000 sample records	System processes without major delay	Pass

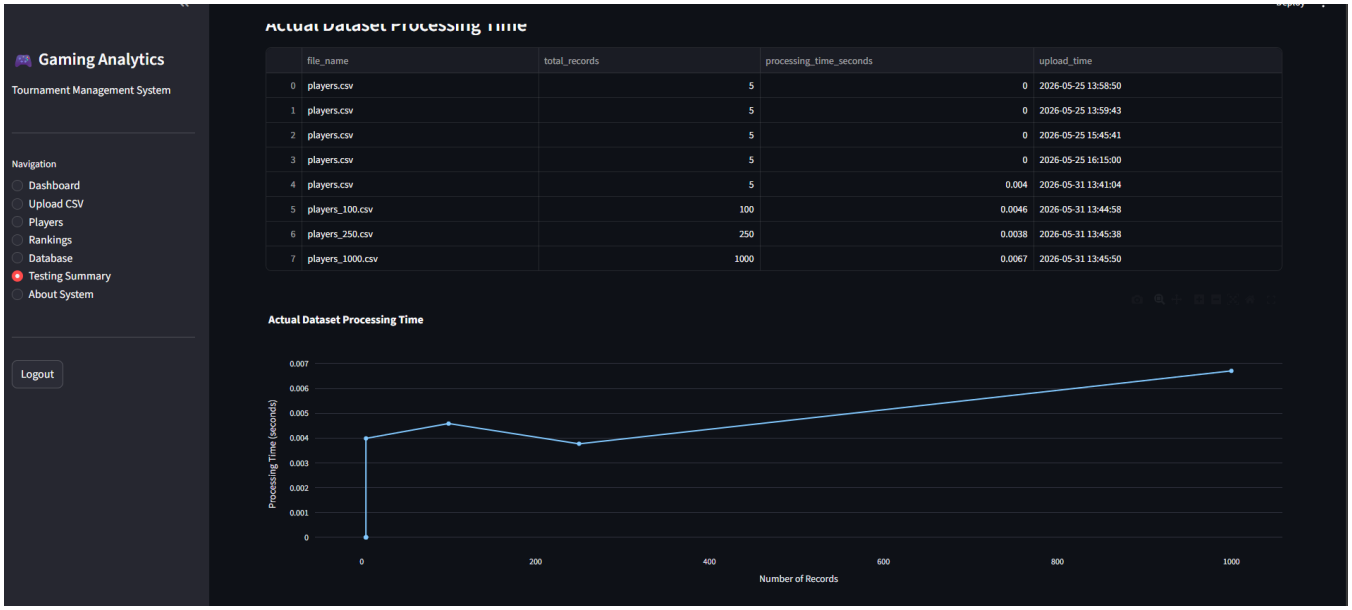


Figure 10.2: Sample Dataset Processing Time

10.3 Results Discussion

- The login function works successfully for system access.
- CSV upload reduces manual data entry work.
- SQLite database stores tournament information in an organized way.
- Ranking generation displays players based on performance points.
- Dashboard charts make the tournament results easier to understand.
- Invalid CSV testing is important because wrong data format can affect the system output.

11. DISCUSSION

11.1 Project Strengths

- Simple and user-friendly dashboard interface.
- Automatic ranking generation reduces manual calculation mistakes.
- SQLite database improves data organization and searching.
- CSV upload makes data import faster.
- Charts help organizers understand player performance visually.
- The system uses low-cost and open-source technologies.

11.2 Challenges Faced

- Database integration required careful table structure and SQL query handling.
- CSV data needed to be formatted correctly to avoid import errors.
- Ranking calculation needed proper sorting logic to avoid wrong ranking positions.
- Dashboard design needed testing to make sure charts and tables were easy to read.
- Preparing the GitHub repository required organizing files into clear folders.

11.3 Project Limitations

- The system currently focuses on local or small tournament usage.
- Real-time online synchronization is not fully implemented.
- Authentication is still basic and can be improved.
- Cloud database integration is not included in the current version.

- AI prediction is suggested as a future improvement but not implemented yet.

11.4 Future Enhancements

- Add online user registration and role-based access control.
- Add real-time match updates and live ranking refresh.
- Use cloud database for multi-device access.
- Develop mobile-friendly interface.
- Add AI-based player performance prediction.
- Add tournament bracket visualization.

12. CONCLUSION

In conclusion, the Smart Gaming Tournament Analytics System successfully provides a simple and efficient solution for managing gaming tournaments. The system reduces manual work by allowing CSV upload, storing data in SQLite database, generating rankings automatically, and displaying analytics charts in a dashboard.

The use of Python and Streamlit makes the system easier to develop and suitable for a Semester 5 IT programming project. SQLite supports structured data storage, while Pandas and visualization libraries help process and present tournament data clearly.

Overall, the project achieves its objectives and fulfills the requirements of BIT1034 Advance Programming. The system has strong potential for future improvements such as real-time updates, cloud database, mobile support, and AI analytics.

13. REFERENCES

References

Atlassian. (2024). Agile methodology. https://www.atlassian.com/agile

Hunter, J. D. (2023). Matplotlib: Visualization with Python. Matplotlib. https://matplotlib.org/

McKinney, W. (2022). Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter (3rd ed.). O'Reilly Media.

Pandas Development Team. (2024). Pandas documentation. https://pandas.pydata.org/docs/

Plotly Technologies Inc. (2024). Plotly Python graphing library. https://plotly.com/python/

Python Software Foundation. (2024). Python documentation. https://docs.python.org/3/

Sommerville, I. (2020). Software engineering (10th ed.). Pearson Education.

Streamlit Inc. (2024). Streamlit documentation. https://docs.streamlit.io/

SQLite Consortium. (2024). SQLite documentation. https://www.sqlite.org/docs.html

APPENDIX A:

SAMPLE CSV DATASET STRUCTURE

Player Name	Team	Country	Wins	Losses	Points
Alex	Team Alpha	Malaysia	10	2	300
Ryan	Team Bravo	Malaysia	8	3	250
Daniel	Team Omega	Singapore	6	4	200
Mira	Team Nova	Malaysia	6	5	180
Farid	Team Titan	Indonesia	5	5	160

APPENDIX B: GITHUB AND SUBMISSION CHECKLIST

GitHub repository link: https://github.com/sahifful/bit1034-b24120060-smart-gaming-tournament-system-

- Final report in PDF format
- Final report in Word format
- Python source code file such as app.py
- SQLite database file or database creation script
- Sample CSV dataset
- Screenshots of running Streamlit application
- GitHub repository link
- Streamlit application files
- Group members understand all parts of the project

Recommended GitHub Folder Structure

```
smart-gaming-tournament-system/
  app.py
  database.py
  requirements.txt
  data/players.csv
  screenshots/dashboard.png
  README.md
  report/BIT1034_report.pdf
```

APPENDIX C: FULL SAMPLE CODE STRUCTURE

```
# app.py
import streamlit as st
import pandas as pd
import sqlite3
import plotly.express as px

conn = sqlite3.connect('tournament.db')

st.title('Smart Gaming Tournament Analytics System')
menu = st.sidebar.selectbox('Menu', ['Dashboard', 'Upload CSV', 'Players', 'Rankings'])

if menu == 'Upload CSV':
    uploaded_file = st.file_uploader('Upload player CSV', type=['csv'])
    if uploaded_file is not None:
        df = pd.read_csv(uploaded_file)
        st.dataframe(df)
        df.to_sql('players', conn, if_exists='replace', index=False)
        st.success('CSV data saved successfully')

if menu == 'Rankings':
    players = pd.read_sql_query('SELECT * FROM players', conn)
    players = players.sort_values(by=['Points', 'Wins'], ascending=False)
    players['Rank'] = range(1, len(players) + 1)
    st.dataframe(players)
    fig = px.bar(players, x='Player Name', y='Points', title='Player Ranking')
    st.plotly_chart(fig, use_container_width=True)
```